Homework 5: Let's get building our online store!

This report collects the screenshots and observations for Homework 5. It covers Part I (the reading post on Piazza), Part II (the Product API built from the OpenAPI spec), Part III (deploying the API to AWS ECS with Terraform), Part IV (load testing with Locust), and Part V (observability with Prometheus, Grafana, and structured logs). The full code, Dockerfile, Terraform, and README are in my forked repository.

Part I: Reading!

I read Chapter 2 of Distributed Systems for Fun and Profit and posted my note in the reading folder on Piazza, plus a comment on another student's post. In short, what I found most informative was the CAP theorem: in a distributed system you can only have two of consistency, availability, and partition tolerance at once, and since network partitions will happen in any real system, the real choice is between consistency and availability during a partition. I also liked the point that consistency is not one single thing, the C in CAP means strong consistency, but weaker models like eventual consistency trade some consistency for availability and speed. What I already had experience with was the idea that nodes do not share memory, which I saw in Homework 2 where two instances of my server each kept their own data and gave different answers.

Link of my note: https://piazza.com/class/mtjky38bxxi60o/post/684

Part II: Overview of the Start of our (Very!) Simple Online Store

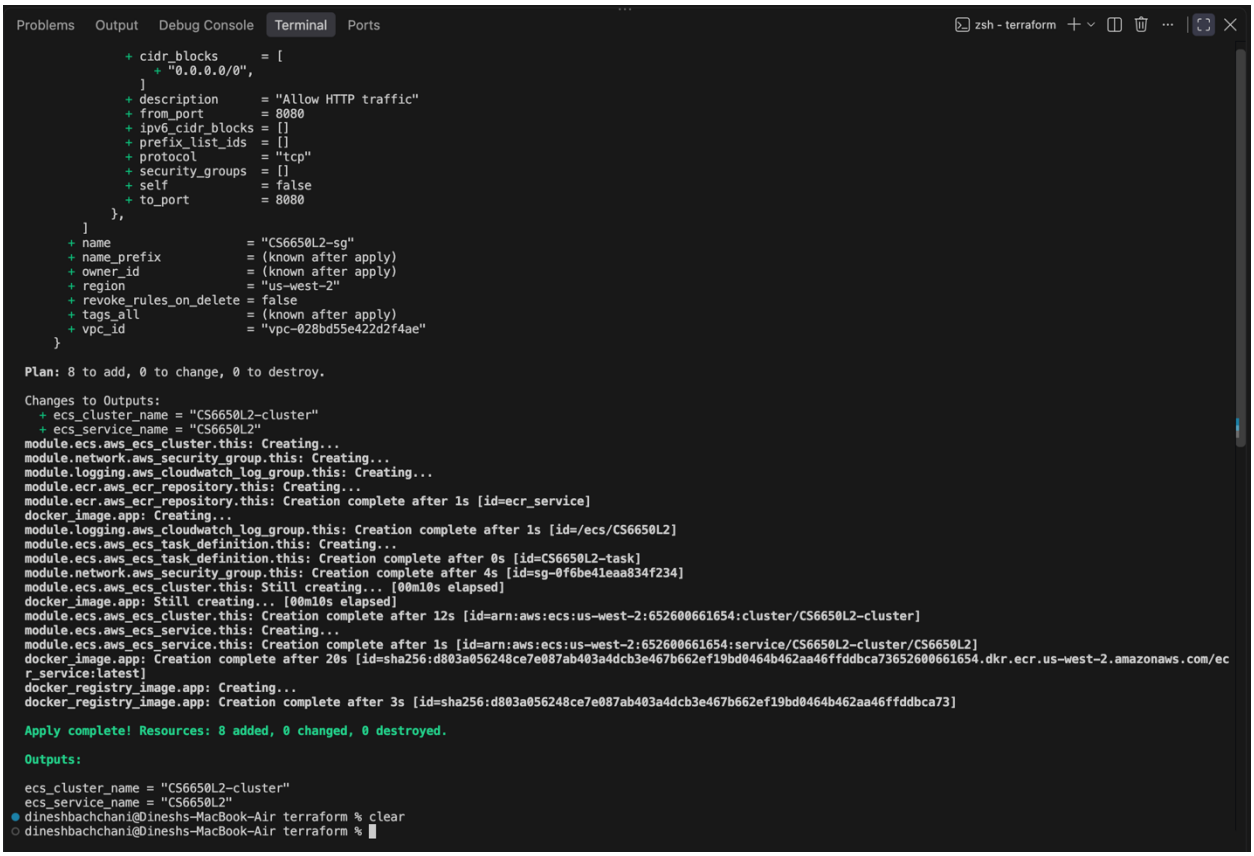
I implemented the Product part of the OpenAPI spec (api.yaml) using Go and the Gin web framework, in product-service.go. There are two endpoints: GET /products/{productId} to fetch a product, and POST /products/{productId}/details to add or update a product. Products are stored in memory in a map, guarded by a read-write mutex so concurrent requests are safe. Input is validated and every response code in the spec is returned: 200, 404, and 400 for GET, and 204, 400, and 404 for POST.

I tested the server locally with go run and inside a Docker container, hitting each endpoint with curl to trigger every response code. The same five response codes are shown against the deployed server in Part III below, so they serve as the response-code evidence for both the local and deployed versions.

Part III: Overview of Our Setup!

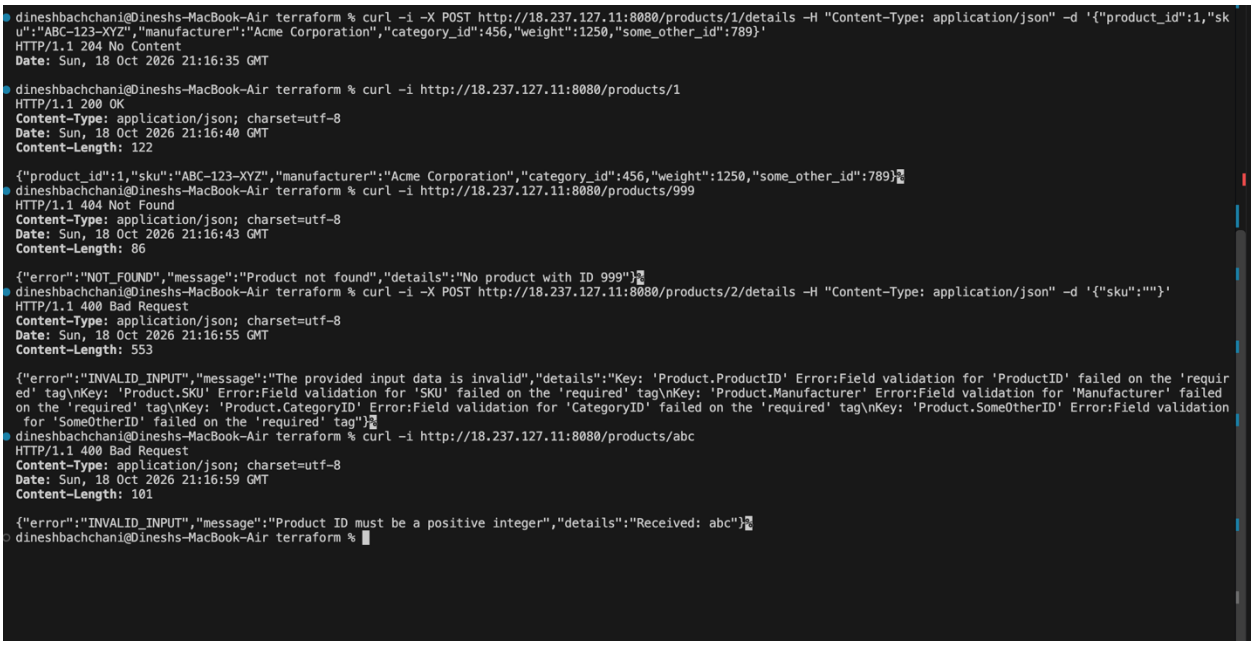
I forked the starter Terraform repository, which deploys a sample server to ECS, and adapted it to deploy my Product API instead. I replaced the server code in src/ with my product-service.go, go.mod, go.sum, and Dockerfile, and made sure the server listens on port 8080 to match the container_port variable. The Terraform is organized into modules (ecr, ecs, logging, network) wired together in main.tf, with provider.tf for the AWS and Docker providers, variables.tf for settings, and outputs.tf for the cluster and service names.

Deploying is just terraform init then terraform apply. Terraform builds my Docker image, pushes it to ECR, and creates the whole stack (networking, cluster, task definition, service) automatically. The apply completed with 8 resources created.



terraform apply created the full stack and deployed the Product API (8 resources, outputs shown).

One issue I had to fix: my Mac builds images for arm64 by default, but Fargate runs amd64, so the first tasks crashed with an exec format error. I fixed it by pinning the platform to linux/amd64 in both the Dockerfile and the Terraform build, after which the task ran cleanly. Once it was running, I got the task's public IP and tested all five response codes against the deployed server.



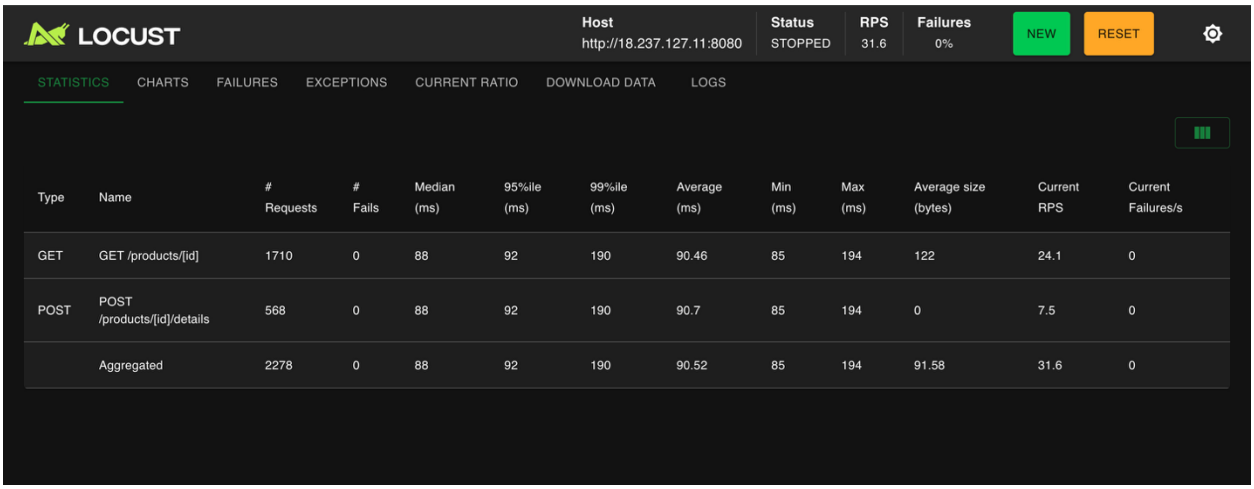
All five response codes working on the deployed API: 204, 200, 404, 400, 400.

Part IV: Testing (yes this looks familiar!)

I used Locust to load test the deployed Product API, with a 3-to-1 GET to POST request mix, the same approach as Homework 3. I ran a baseline test, a stress test, and a FastHttpUser comparison.

Baseline: 50 users

At 50 users the server handled about 31.6 requests per second with zero failures and a median response time of about 88 ms. This is comfortable, the server is not stressed at all.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	GET /products/[id]	1710	0	88	92	190	90.46	85	194	122	24.1	0
POST	POST /products/[id]/details	568	0	88	92	190	90.7	85	194	0	7.5	0
	Aggregated	2278	0	88	92	190	90.52	85	194	91.58	31.6	0

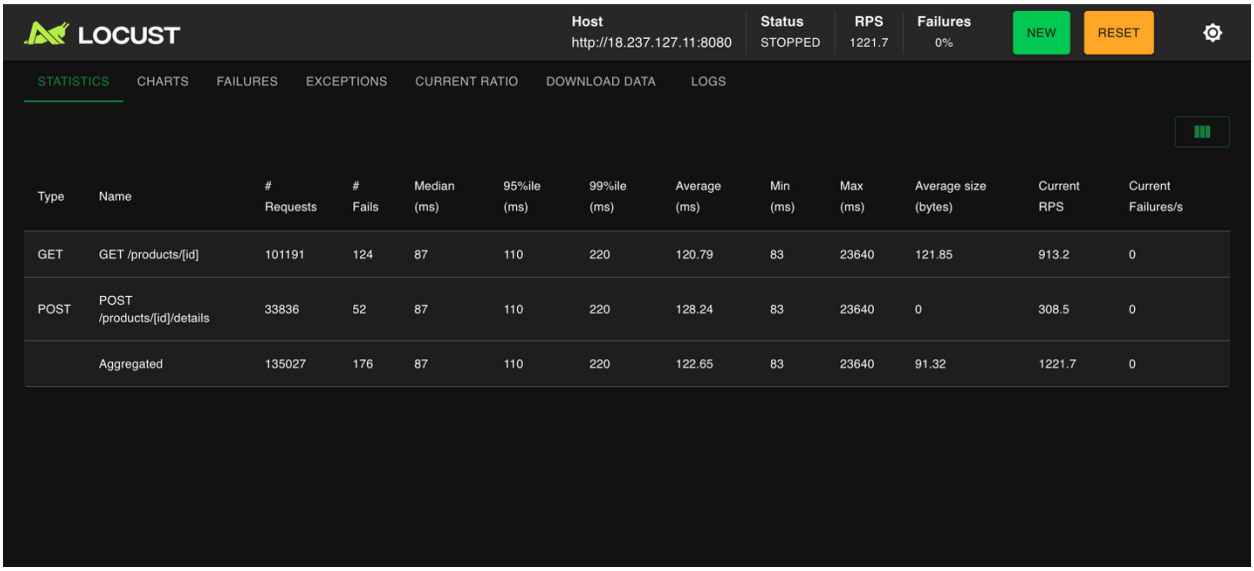
Baseline 50 users: ~31.6 req/sec, 0 failures.



Baseline 50 users: throughput and response times stay low and steady with no failures.

Stress test: 2000 users

To find where the server struggles, I pushed it to 2000 users. Throughput plateaued at about 1221 requests per second and could not go higher, some requests started failing (176 total), and the maximum response time spiked to about 23 seconds even though the median stayed around 87 ms. This is the point where the server starts to struggle. The single Fargate task has only 0.25 vCPU, so once it hits its throughput ceiling, extra requests queue up (causing the huge max latency) and some get dropped (the failures). The fix would be to scale horizontally with more tasks behind a load balancer or give the task more CPU and memory.



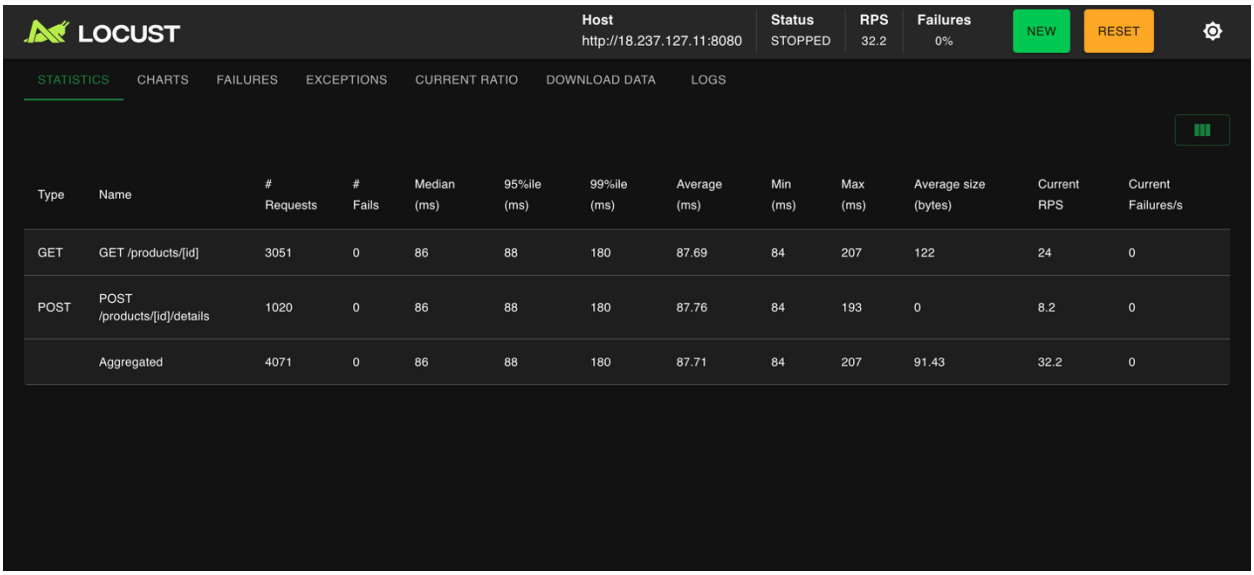
Stress test 2000 users: throughput plateaued at ~1221 req/sec, 176 failures, max latency ~23 seconds.



Stress test: RPS climbs then plateaus flat even as users keep rising to 2000, the throughput ceiling.

HttpUser vs FastHttpUser

I reran the 50-user baseline with FastHttpUser, which uses a faster C-based HTTP client, to compare against the default HttpUser. The result was nearly identical: about 32 requests per second and similar response times. At this modest load the client is not the bottleneck, so the faster client has nothing to speed up, the response times are dominated by the server and the network round trip, not the client library. FastHttpUser only makes a difference when the load generator itself is the limiting factor, for example when pushing very high load from one machine. This matches what I found in Homework 3.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	GET /products/[id]	3051	0	86	88	180	87.69	84	207	122	24	0
POST	POST /products/[id]/details	1020	0	86	88	180	87.76	84	193	0	8.2	0
	Aggregated	4071	0	86	88	180	87.71	84	207	91.43	32.2	0

FastHttpUser 50 users: ~32 req/sec, nearly identical to HttpUser.



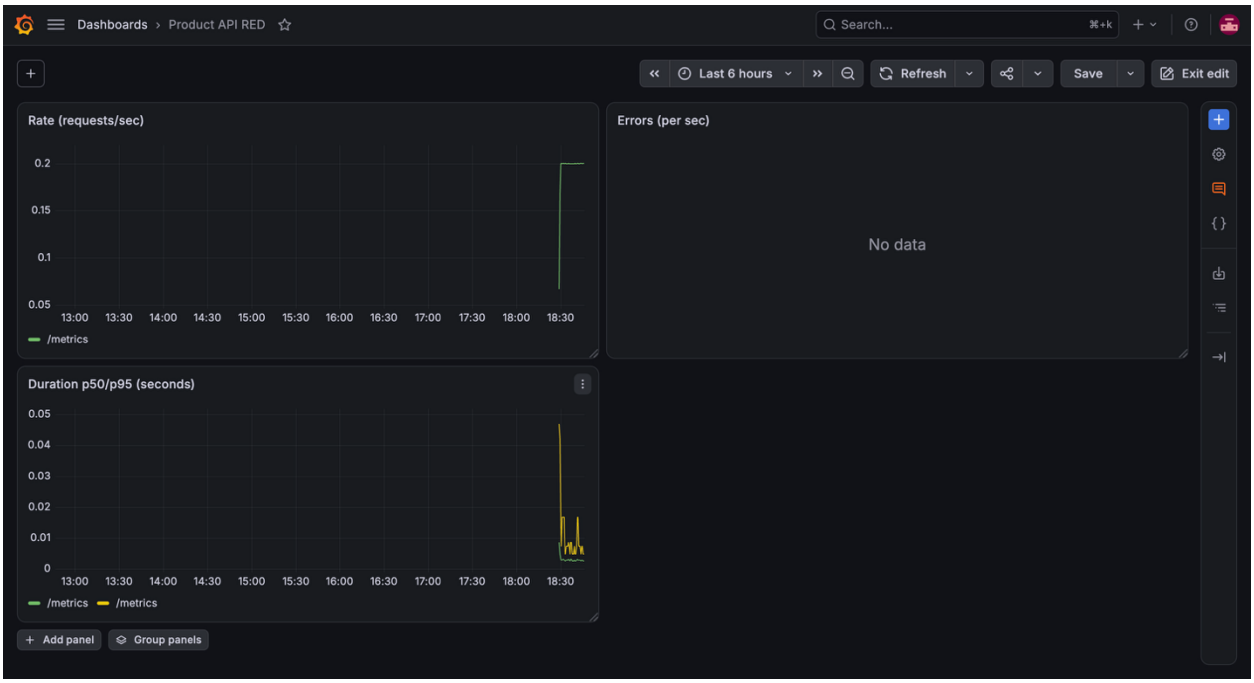
FastHttpUser: steady throughput and response times, essentially the same as the default client.

Part V: Observability — Know Your Vitals

I added observability to the server so I can see its behavior under load. The server now exposes a /metrics endpoint for Prometheus, tracking the RED signals: Rate (a counter of total requests), Errors (a counter of non-2xx responses), and Duration (a histogram of request latency). It also emits one structured JSON log line per request with the method, path, status code, and duration. I ran Prometheus and Grafana locally with docker-compose, with Prometheus scraping the server every 5 seconds and Grafana showing a dashboard with one panel per RED signal.

The dashboard: idle vs loaded

With no real traffic, the dashboard is essentially flat, only the Prometheus scrapes show up, and the Errors panel has no data because nothing is failing.



Idle state: the dashboard is flat with only scrape traffic, and no errors.

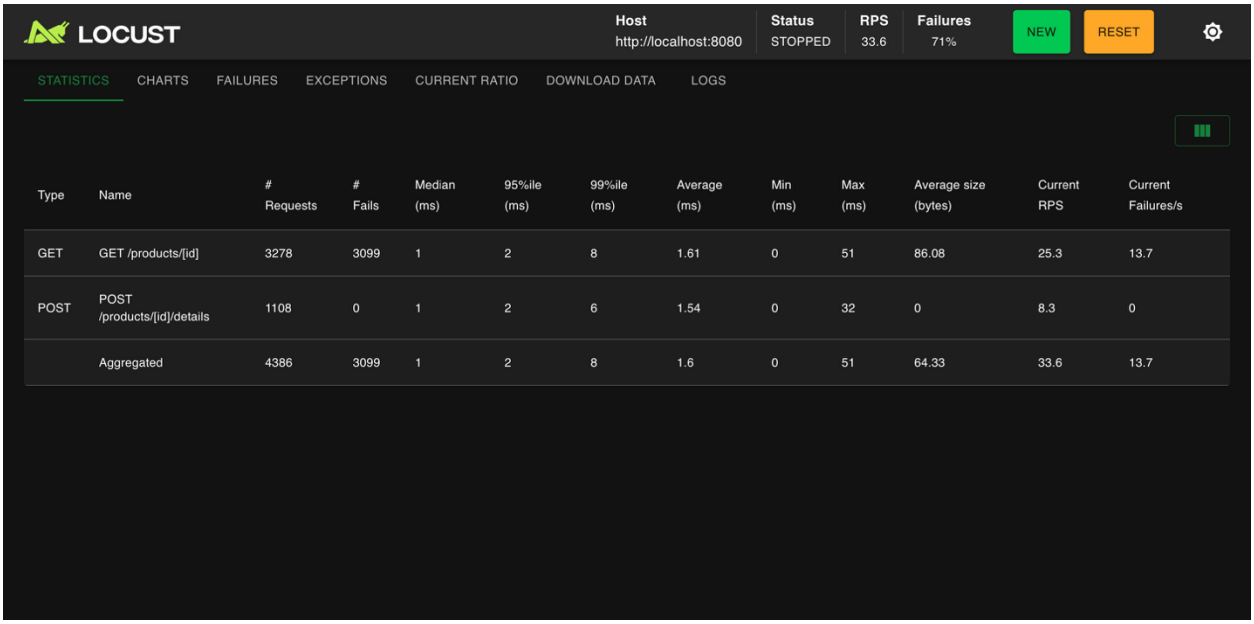


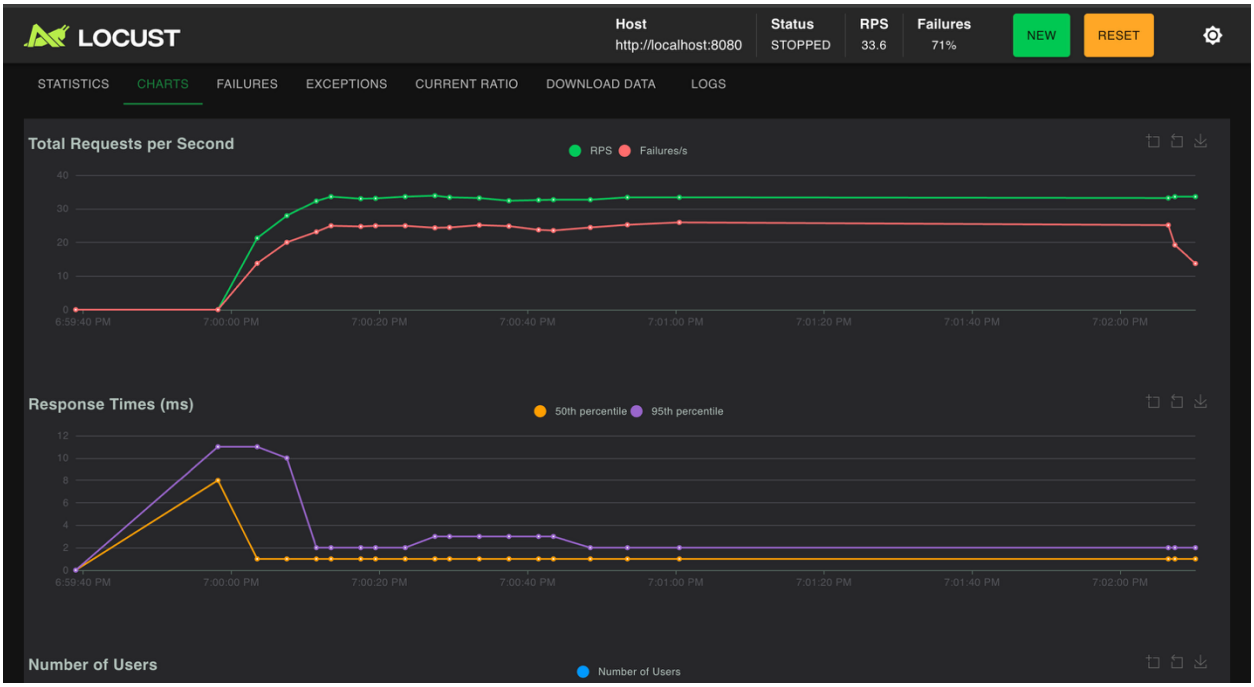
When I drove load against the local server with Locust, all three panels came alive: the Rate panel jumped up for both endpoints, the Duration panel showed the request latencies, and the Errors panel lit up. The errors appeared because the GET load hit products that did not exist in the freshly started local instance, so the server correctly returned 404s. This is a good demonstration that the observability setup surfaces error conditions in real time.



Loaded state: all three RED panels light up when load starts, flat before, active after.

The same run driven from Locust is shown below, the failures here are the 404s that the Errors panel detected.





The local Locust run that drove the dashboard; the failures are the 404s the Errors panel captured.

Structured logs

Alongside the dashboard, the server emits one JSON log line per request with the same RED data (method, path, status, duration). This is useful for after-the-fact debugging when no dashboard was open at the time. A short excerpt from the same run:

{"duration_ms":0,"method":"POST","path":"/products/:productId/details","status":204,"time":"2026-10-18T19:02:08-04:00"}{"duration_ms":0,"method":"GET","path":"/products/:productId","status":200,"time":"2026-10-18T19:02:08-04:00"}{"duration_ms":0,"method":"GET","path":"/products/:productId","status":200,"time":"2026-10-18T19:02:08-04:00"}{"duration_ms":1,"method":"GET","path":"/metrics","status":200,"time":"2026-10-18T19:02:12-04:00"}{"duration_ms":2,"method":"GET","path":"/metrics","status":200,"time":"2026-10-18T19:02:27-04:00"}

Dashboard vs logs

I would look at the Grafana dashboard for a live, at-a-glance view during load, to spot trends like rising error rates or latency in real time. I would look at the logs after the fact to debug a specific request, since each log line has the exact endpoint, status, and duration that the aggregated dashboard cannot show.

Cleanup

After collecting all the results, I stopped the local server, Locust, and the Prometheus/Grafana containers, and ran terraform destroy (plus a forced ECR repo delete) to tear down all the AWS resources so nothing kept running against my budget.